

AMF-Placer

High-Performance Analytical Mixed-size Placer for FPGA

Day 1: 为什么需要 Mixed-size Placement?

从 FPGA 资源层次、Macro 定义、核心挑战到 AMF-Placer 全流程

今天不追求“把算法公式全学完”。Day 1 的目标是先建立一张正确的心智地图：

- FPGA placement 到底在放什么；
- 为什么“macro”会让传统 FPGA analytical placer 失效；
- mixed-size placement 的数学问题是什么；
- AMF-Placer 的 7 个阶段各自负责解决什么。



目录

1	今天先抓住论文的主线	2
1.1	Day 1 的五个问题	2
2	先建立 FPGA placement 的物理直觉	2
2.1	从 Device 到 Site, 再到 BEL	2
2.2	Instance 才是 placer 操作的对象	3
3	论文最重要的定义: Standard Cell 与 Macro	3
3.1	Standard cell: 最小、不可再拆的表示单元	3
3.2	为什么“大小”不是重点, “内部关系”才是重点	4
4	为什么 Mixed-size Placement 比普通 FPGA Placement 难	4
4.1	挑战 1: Macro 的连接密度很高, 轻微移动也会扭曲线长模型	4
4.2	挑战 2: Macro 与普通 cell 可能发生“细粒度共享”	5
4.3	挑战 3: Legalization 从 1-to-1 变成 1-to-many	5
4.4	挑战 4: Macro 数量也很大, 不能把它们当少数特殊对象处理	5
5	Mixed-size Placement 的数学问题是什么?	6
5.1	Hypergraph 建模	6
5.2	HPWL: 今天只需要理解几何意义	6
6	AMF-Placer 的 7 阶段全景图	7
6.1	阶段 1: SA-based Initial Placement	7
6.2	阶段 2: Quadratic Placement	7
6.3	阶段 3: Cell Spreading	7
6.4	阶段 4: Resource Demand/Supply Adjustment	7
6.5	阶段 5: Progressive Macro Legalization	7
6.6	阶段 6: Incremental Packing	7
6.7	阶段 7: Final Packing	8
7	Lower-bound 与 Upper-bound: 后续读算法的钥匙	8
8	把 Day 1 串成一条因果链	9
8.1	论文贡献先做第一层分类	9
9	Day 1 自测: 建议先口头回答, 再看答案	9
10	后续学习路线	10

1 今天先抓住论文的主线

一句话概括: AMF-Placer 的核心不是“又做了一个 FPGA placer”，而是把过去分析式 FPGA placer 很少认真处理的 **multi-cell macro** 纳入统一的 **mixed-size placement** 流程，并针对 macro 带来的初始布局、二次优化、扩散、合法化与 **packing** 问题逐段改造。

论文指出，现代 FPGA 设计在综合/上游优化后，关键路径的部分逻辑可能被预先组织成具有**形状约束和资源约束**的多单元 macro。这些 macro 是可移动的，但内部单元之间的相对位置不能随意破坏，因此传统“每个 instance 占一个 BEL”的建模不够用了。论文将其称为 FPGA 上的 **mixed-size placement**。
(论文摘要、Sec. I)

1.1 Day 1 的五个问题

1. FPGA placement 中的 site、BEL、instance 分别是什么？
2. 论文里的 **standard cell** 和 **macro** 到底如何区分？
3. 为什么 macro 不是“把几个 cell 当成一个大矩形”这么简单？
4. **mixed-size placement** 的目标函数和约束是什么？
5. AMF-Placer 为什么要设计成 7 个阶段，而不是一次性求解？

读完今天的内容，你至少应该能够不看论文，自己解释：

macro $\Rightarrow$ 高连接密度 + 多资源占用 + 1-to-many legalization + 稀疏合法位置

这条因果链是后续所有算法设计的出发点。

2 先建立 FPGA placement 的物理直觉

2.1 从 Device 到 Site，再到 BEL

论文 Fig. 1 用 Xilinx UltraScale 设备说明了 FPGA 的资源层次。最外层是二维的 FPGA device；device 内有不同种类的 **site**，例如 CLB、DSP、RAM；一个 CLB site 内又包含更细粒度的 **BEL (Basic Element of Logic)**，例如 LUT、FF、CARRY、MUX。（论文 Fig. 1, Sec. I-A）

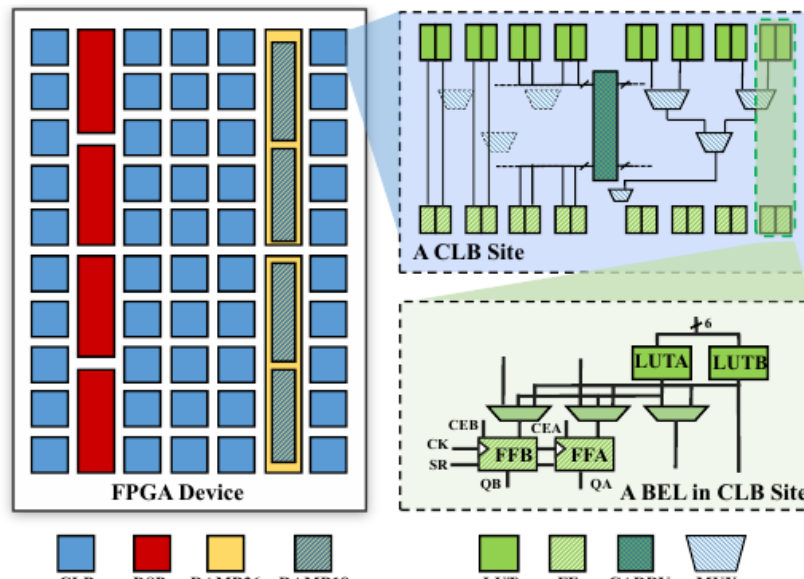


图 1 原论文 Fig. 1 的主要图像：Xilinx UltraScale FPGA device、CLB site 与内部 BEL。为了便于阅读，这里从用户提供的论文中截取图像。

可以把层次理解为：



Placement 不是只决定一个连续坐标。分析式优化阶段可以先用连续坐标近似，但最终必须满足器件架构：LUT 去 LUT 资源、DSP 去 DSP column、BRAM 去 BRAM column，并且某些 macro 还要求连续行、同一列或特定内部路由关系。

2.2 Instance 才是 placer 操作的对象

逻辑综合产生 netlist。论文将 netlist 中要放置的对象视为 **instances**，网络连接视为 **nets**。placer 的任务可以简化为：给每个可移动 instance 一个位置，同时满足资源合法性，并尽量降低线长、拥塞和时序代价。论文在 Day 1 的数学建模中重点使用 HPWL 作为目标。



placer 决定这些 instance 在 FPGA 上的物理位置

3 论文最重要的定义：Standard Cell 与 Macro

3.1 Standard cell：最小、不可再拆的表示单元

论文明确给出定义：**standard cell** 是设计 netlist 中最小、不可分、可表示的组件；**macro** 是多个 standard cells 的固定组合，并带有内部相对位置约束。这里的“standard cell”是论文在 FPGA placement 语境里的术语，不要机械等同于 ASIC standard-cell library。（论文 Sec. I-A）

论文给了三个非常关键的例子：

- 1 个 MUX 与连接到它的 2 个 LUT，应作为一个 macro；
- 一个没有和其他 BRAM cascade 的 BRAM，可视为 standard cell；
- 3 个级联 DSP，应视为一个 macro。

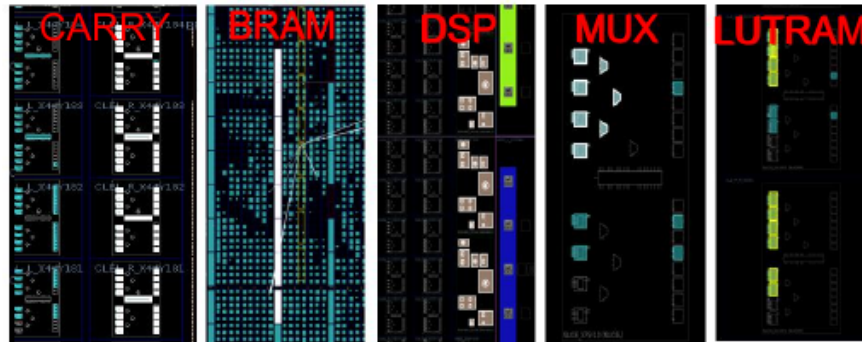


图 2 原论文 Fig. 2 的核心图像：CARRY、BRAM、DSP、MUX、LUTRAM 等不同类型的 macro。图中高亮部分体现了 macro 的形状/资源约束。

3.2 为什么“大小”不是重点，“内部关系”才是重点

先看一个极简示意：



关键：macro 内部可能要求连续占用多个 site、同列放置、保持固定顺序，甚至为内部 routing 预留 LUT/FF 槽位。

所以它不是一个“面积更大的独立 cell”，而是一个带结构约束的组合对象。

论文进一步说明，macro 的核心单元可能是 CARRY、MUX、LUTRAM、DSP 或 BRAM，并可能附带直接相连的 LUT/FF。比如 CARRY macro 可能因为内部路由需求占用额外的 route-thru LUT，某些 FF 槽也可能因为布线资源冲突而不可用。（论文 Sec. II-A）

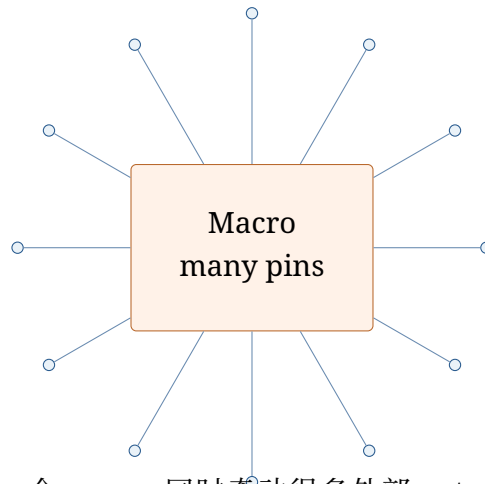
当你以后看到 AMF-Placer 的 **resource demand/supply adjustment**、**progressive legalization**、**pseudo net to legal locations** 时，都要回到这里：**macro 不仅占面积，它还占“特定形状的特定资源”。**

4 为什么 Mixed-size Placement 比普通 FPGA Placement 难

论文在 Sec. I-B 给出了四类直接挑战。这里逐条解释，不急着进入算法。

4.1 挑战 1：Macro 的连接密度很高，轻微移动也会扭曲线长模型

论文举例：一个 CARRY macro 可能包含 27 个 LUT、25 个 FF 和 3 个 CARRY，并与 macro 外部连接超过 200 条 nets。这样一个对象只移动一点点，可能同时改变大量 net 的几何关系。（论文 Sec. I-B）



同一个 macro 同时牵动很多外部 nets，因此 analytical model 对 macro 位移更敏感。

这也是论文后面在 pseudo-net 权重中引入 **pinNum** 的根本动机：高连接密度实例要“走慢一点”，避免每轮 QP 解产生过大跳动。这个公式我们放到 Day 2 再推导。

4.2 挑战 2：Macro 与普通 cell 可能发生“细粒度共享”

论文指出，一些 macro 可以和其他 instance 共享 CLB 的部分资源，因此 mixed-size placement 中的“overlap”并不一定像 ASIC 大 macro 那样完全非法。换句话说，合法性是资源槽级别的，而不只是几何矩形是否重叠。

如果把 FPGA macro 直接当成 ASIC hard macro，用一个不可重叠的大矩形建模，会过度保守；但如果完全拆散 macro，又会破坏形状与内部路由约束。AMF-Placer 必须在两者之间找到可优化的中间表达。

4.3 挑战 3：Legalization 从 1-to-1 变成 1-to-many

传统情况可以近似理解为：

1 instance $\rightarrow$ 1 legal site

而 macro 的情况更像：

1 macro $\rightarrow$ 一组连续/相关的 legal sites

这直接改变了 legalization 的组合复杂度。论文强调，以往逐个实例、贪心式的 legalization 容易陷入局部最优；对于 macro，还必须同时决定列、起始行、连续长度以及与其他 macro 的冲突关系。

4.4 挑战 4：Macro 数量也很大，不能把它们当少数特殊对象处理

论文以 MiniMap2 为例，指出设计中可有 8782 个 macros、499104 个 standard cells；此前 HeAP 相关 benchmark 中 macro 数量远少得多。也就是说，macro 不是“少数例外”，而会成为 placement 规模的一部分。（论文 Sec. I-B）

把四个挑战压缩成四个关键词：

高连接密度 / 资源共享 / 1-to-many / 大规模

只要后面某个技术看不懂，先问：它主要在解决这四个问题中的哪一个？

5 Mixed-size Placement 的数学问题是什么？

5.1 Hypergraph 建模

论文将设计写成超图

$$H = (V, E),$$

其中

$$V = \{v_1, v_2, \dots, v_n\}$$

表示 instances,

$$E = \{e_1, e_2, \dots, e_m\}$$

表示 nets。每个 instance v_i 的中心位置记为 (x_i, y_i) 。instance 可以是 standard cell，也可以是 macro；还可以根据约束分为 movable 或 fixed。（论文 Sec. II-B）

为什么用超图而不是普通图？因为一个 net 往往可以同时连接多个 pins/instances，而不是只连接两个顶点。

5.2 HPWL：今天只需要理解几何意义

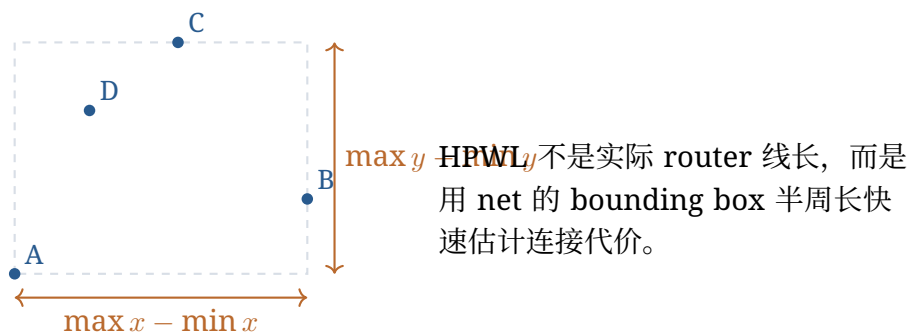
论文把常用的 placement 目标写成所有 nets 的 Half-Perimeter Wirelength 之和。对 net e ，其二维 HPWL 可以写成：

$$\text{HPWL}(e) = \left(\max_{i \in e} x_i - \min_{i \in e} x_i \right) + \left(\max_{i \in e} y_i - \min_{i \in e} y_i \right).$$

于是总目标近似为

$$\min_{x, y} \sum_{e \in E} \text{HPWL}(e),$$

但必须满足：每个 instance 最终位于该资源类型允许的合法位置，且 macro 的内部/形状/区域约束不能被破坏。



论文原意

论文的关键表述是：placer 要决定所有 movable instances 的位置，使总 HPWL 尽量小，同时满足 technology 与 region constraints。后续的 quadratic placement，就是为了把不可直接微分/不便直接求解的 HPWL 换成可高效求解的二次近似。

6 AMF-Placer 的 7 阶段全景图

论文 Fig. 3 是 Day 1 最值得记住的一张图。它告诉我们：AMF-Placer 并不是单个优化器，而是一套 **lower-bound optimization + upper-bound legalization/spreading + packing** 交替收敛的流程。（论文 Fig. 3, Sec. II-C）

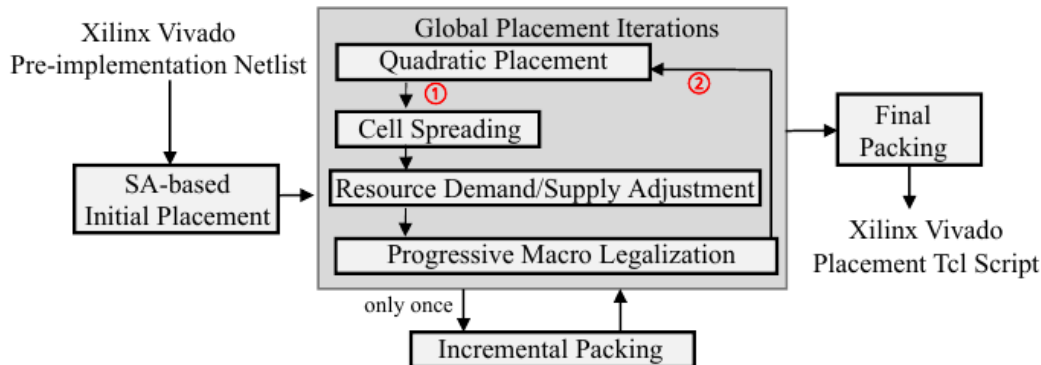


图3 原论文 Fig. 3 的核心工作流图。全局布局循环中反复进行 Quadratic Placement、Cell Spreading、Resource Demand/Supply Adjustment 与 Progressive Macro Legalization，并在合适阶段进行 Incremental Packing。

6.1 阶段 1: SA-based Initial Placement

与许多从随机位置开始的 analytical placer 不同，AMF-Placer 先做基于 cluster 的 simulated annealing 初始布局。其目的不是靠 SA 完成最终 placement，而是利用 SA 的随机搜索能力，跳出 analytical model 容易陷入的局部极小区域。

6.2 阶段 2: Quadratic Placement

QP 的职责是：在连续坐标空间里快速寻找更低线长的 **lower-bound placement**。论文同时把 inter-connection density 和 legalization 信息通过 adaptive pseudo nets 注入二次模型。

6.3 阶段 3: Cell Spreading

QP 只顾线长时，instances 会挤在一起。Cell Spreading 根据区域的资源 demand 与 supply，把 overflow region 中的实例往外扩散，得到更可实现的 **upper-bound placement**。

6.4 阶段 4: Resource Demand/Supply Adjustment

FPGA 的 LUT/FF packing 与 routing feasibility 会影响“一个区域到底还能装多少东西”。论文因此动态调整某些资源的 demand/supply，而不是把几何面积当成固定不变的容量。

6.5 阶段 5: Progressive Macro Legalization

这是论文最有辨识度的部分之一。早期不强行把 macro 一次性钉死在精确合法位置，而是先 rough legalization，再随着 placement 变稳定逐步过渡到 exact legalization，并用 pseudo nets 将 macro 拉向候选 legal anchors。

6.6 阶段 6: Incremental Packing

全局布局过程中，部分 LUT/FF 会提前配成 LUT-FF macro、FF-FF macro，目的是减少问题规模并尽早识别 CLB 内部连接。

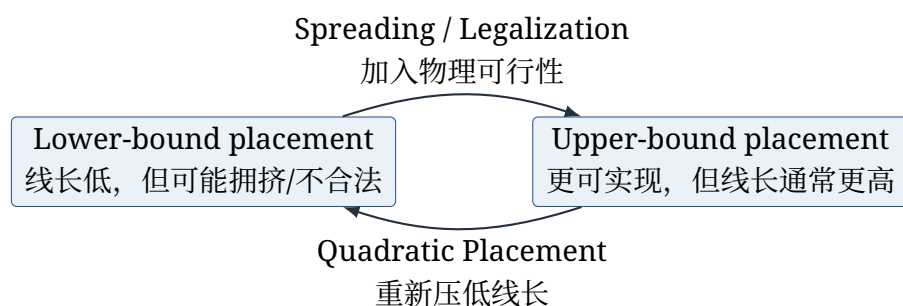
6.7 阶段 7: Final Packing

全局迭代结束后，将 LUT、FF、MUX、CARRY 等真正映射到具体 CLB site，实现最终的 packing / legalization。

阶段	主要问题	你可以先记住的关键词
Initial Placement	避免糟糕初值、突破局部最优	SA + clustering
Quadratic Placement	降线长、产生连续坐标解	lower-bound
Cell Spreading	消除局部资源 overflow	rough legalization
Demand/Supply Adj.	反映 packing/routing 可行性	effective capacity
Macro Legalization	处理 1-to-many macro 合法化	rough → exact
Incremental Packing	提前形成局部 packing 结构	LUT-FF / FF-FF
Final Packing	映射到具体 site/BEL	discrete legal solution

7 Lower-bound 与 Upper-bound: 后续读算法的钥匙

论文把 Quadratic Placement 生成的、线长更低但可能不合法/拥挤的解称为 **lower-bound placement**；把 spreading/legalization 后、更接近实际资源限制、通常线长更高的解称为 **upper-bound placement**。当二者的 HPWL 差异逐渐缩小，并且 macros 已精确合法化，流程趋于收敛。（论文 Sec. II-C）



可以把迭代想象为两个目标之间的“拉扯”：

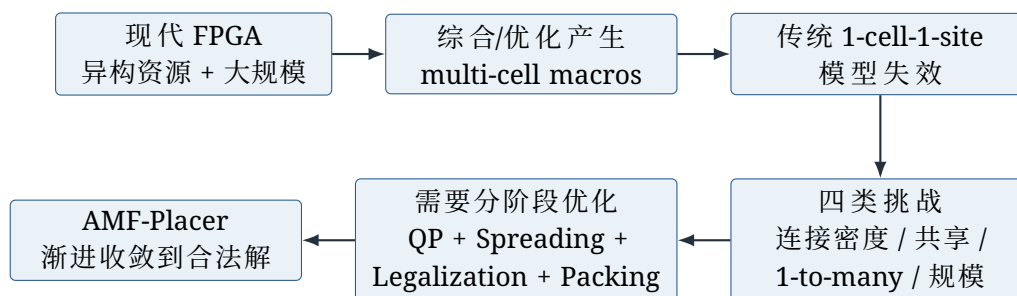


如果只做 QP：线长很好，但大量 cell 可能重叠或挤在资源不足区域。

如果只做 legalization：位置合法，但可能产生巨大 displacement，使线长恶化。

AMF-Placer 的核心思想是让两者**渐进地靠拢**，而不是在最后一刻突然把一个连续解“硬掰”成合法解。

8 把 Day 1 串成一条因果链



这就是今天真正要记住的“论文逻辑”，而不是某个公式编号。

8.1 论文贡献先做第一层分类

论文列出的技术点很多，但可以先按问题域分类：

- **初值**：SA-based cluster-level initial placement;
- **线长优化**：interconnection-density-aware pseudo-net weights;
- **扩散**：utilization-guided spreading window、resource supply fluctuation、forgetting-rate location update;
- **Macro 合法化**：progressive macro legalization;
- **工程性能**：各阶段并行化。

今天只需要知道“它们为什么存在”。Day 2 之后再进入“它们具体怎么算”。

9 Day 1 自测：建议先口头回答，再看答案

1. 为什么论文说 AMF-Placer 处理的是 **mixed-size placement**?
2. 一个单独的 BRAM 为什么可以是 standard cell，而 3 个 cascaded DSP 却是 macro?
3. 为什么 macro 的 legalization 不能简单沿用 1-to-1 matching?
4. HPWL 在 placement 里扮演什么角色？它是不是最终真实 routed wirelength?
5. lower-bound placement 和 upper-bound placement 的本质区别是什么?
6. Progressive Macro Legalization 为什么要“progressive”，而不是一开始就 exact legalization?

1. 因为设计同时包含单个 standard cells 与跨多个资源位置、带形状约束的 macros，放置对象尺寸/占用模式不同。
2. 关键不在物理尺寸，而在是否由多个 standard cells 组成并必须保持内部相对位置/级联结构。

3. 一个 macro 需要一组连续或相关的 sites, 选择一个位置会同时占用多个资源并影响其他 macro, 属于 1-to-many 组合合法化。
4. HPWL 是快速的几何线长代理目标, 用于指导 placement; 它不是最终 router 的真实线长。
5. lower-bound 更偏连续优化与低线长, upper-bound 更偏资源可实现性与粗合法化。
6. 早期 placement 密度高、位置不稳定, 过早精确合法化会产生大位移并把优化锁进差的局部最优; 因此先粗后精。

10 后续学习路线

为了保持“每次只吃透一块”, 建议按下面节奏继续:

Day 2	Initial Placement + Quadratic Placement: 为什么随机初值不够; PaToH clustering; SA cost; B2B quadratic model; pseudo net; 为什么权重要乘 pinNum。
Day 3	Cell Spreading: overflow bin; window expansion; resource fluctuation; macro deadlock; two-phase spreading; forgetting rate。
Day 4	Progressive Macro Legalization: rough matching; candidate site; min-cost bipartite matching; pseudo anchor; exact legalization; 列内 DP。
Day 5	Packing + Experiments: incremental/final packing; hash-based candidate packing; Tech1-Tech5 消融; 并行加速; 与 Vivado 的差距和局限。

今天的结束语

今天的重点不是记住每个阶段的实现细节, 而是先建立一个稳定的解释框架: **AMF-Placer** 的每一个技巧, 都来自 **macro** 破坏了传统 **analytical FPGA placement** 的某个假设。

下一天我们会进入论文最数学化的第一部分: **SA 初始布局 + Quadratic Placement + pseudo-net 权重**, 并把 Eq. (1)–(8) 一步一步拆开。